

第14回 行列と解析：連立方程式を「一撃で解く」

💡 ベクトルを並べた「データベース」を使って、複雑な回路を全自動で解く 💡

🕒 授業時間の目安（100分授業・正味85分）

時間	内容
0〜5分	今日のゴール、前回のベクトルとのつながり
5〜20分	行列とは何か？：ベクトルを積み重ねた「表」・「行」と「列」の覚え方
20〜35分	🌟 行列の掛け算ルール：前の行（横）× 後ろの列（縦）
35〜50分	現場での役割：連立方程式を行列の形 $A\vec{x} = \vec{b}$ に仕分ける
50〜65分	💻 ワーク1： <code>np.linalg.solve()</code> を使ってColabで一撃で解く
65〜85分	⚡ 【実務応用】ワーク2・3：電気回路のノード電圧を自動計算する
+α	🕒 エクストラワーク & 💡 知的好奇心コラム（時間が余った場合）

🎯 今日のゴール

今日の授業の目標は以下の5つです。

1. **行列**を「ベクトル（矢印）をたくさん並べて作ったデータベース」として理解する。
2. 漢字の形をヒントに、「**行（横）**」と「**列（縦）**」を絶対に書き間違えないようになる。
3. 行列の掛け算ルール「**前の行（横）** × **後ろの列（縦）**」の動きをマスターする。
4. 面倒な連立方程式を、行列の形（ $A\vec{x} = \vec{b}$ ）に整理できるようになる。
5. Google Colab の `np.linalg.solve()` を使い、未知数がいくつあっても一瞬で解ける全自動の快感を体験する。

0. 前回とのつながり：ベクトルを「並べた」ものが行列である

昨日の授業では、ベクトル（矢印）を学びました。たとえば、マッチングアプリのユーザーの性格を「（アクティブ度, まったり度）」という2つの数字のセット（ベクトル）で表しましたね。

- ユーザーAさん： $\vec{u} = \begin{bmatrix} 4 & 2 \end{bmatrix}$
- よしこさん： $\vec{v} = \begin{bmatrix} 5 & 1 \end{bmatrix}$
- たかし君： $\vec{w} = \begin{bmatrix} 2 & 4 \end{bmatrix}$

これは「1人分のデータ = 1本の矢印」です。では、あなたがこのアプリの開発者だとして、**全ユーザーのデータを1つの大きな塊としてまとめて管理・計算したいとき**はどうすればいいのでしょうか？

答えは簡単です。**全員のベクトルを上から順番に積み重ねて「表」にしていしまえばいい**のです。

$$M = \begin{bmatrix} 4 & 2 \\ 5 & 1 \\ 2 & 4 \end{bmatrix} \begin{array}{l} \leftarrow \text{ユーザーAさんのベクトル (行ベクトル)} \\ \leftarrow \text{よしこさんのベクトル (行ベクトル)} \\ \leftarrow \text{たかし君のベクトル (行ベクトル)} \end{array}$$

このように、思考や特徴を表すベクトルを複数人分並べて作った「数字の格子状の表」のこと。これを数学では『**行列 (ぎょうれつ)**』と呼びます。見た目は難しそうですが、本質はExcelのデータシートや、データベースのテーブルと全く同じです。

1. 脳内に一生焼き付ける「行」と「列」の覚え方

行列を扱う上で、全理工系学生が一度はこんがらがって絶望するのが「**行 (ぎょう) って横だっけ？ 縦だっけ？**」という問題です。今日、この瞬間に完璧に脳に焼き付けましょう。漢字の「形」がそのまま答えになっています。

- 「**行 (横)**」：漢字の「行」の右側 (つくり) を見てください。**横棒**が2本力強く走っていますね。だから **行は「横方向」** です。
- 「**列 (縦)**」：漢字の「列」の右側 (りっとう) を見てください。**縦棒**が2本 (「リ」の形) 並んでいますね。だから **列は「縦方向」** です。

【行 (ぎょう)】

--
--

(横の並び)

【列 (れつ)】

||
||

(縦の並び)

さっきのマッチングアプリの行列 M で言えば、

- 「**行 (横)**」を見る：ユーザーごとの特徴 (Aさんのベクトル、よしこさんのベクトル...) が分かります。
- 「**列 (縦)**」を見る：項目ごとのデータ (1列目は全員のアクティブ度、2列目は全員のまったり度) が分かります。

2. 行列の掛け算は「前の行 (横) × 後ろの列 (縦)」

これから連立方程式を解くために、行列とベクトルの「掛け算 (積)」を使います。行列の掛け算には、独特で泥臭いルールがあります。合言葉は「**前の行 (横) かける 後ろの列 (縦)**」です。

実際に、次の掛け算がどう計算されるか、指を動かしながら目で追ってみてください。

$$\begin{bmatrix} 2 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

左側にあるのが「前の行列」、右側にあるのが「後ろのベクトル」です。

1段目 (上の段) の計算

前の行列の **1行目 (上の横並び [2, 3])** と、後ろのベクトルの **1列目 (縦並び [x, y])** を、先頭から順番にペアにして掛け算し、最後に足し合わせます。

$$\begin{bmatrix} 2 & 3 \\ \cdot & \cdot \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow (2 \times x) + (3 \times y) \Rightarrow 2x + 3y$$

2段目（下の段）の計算

今度は、前の行列の **2行目（下の横並び [4, -1]**）と、後ろのベクトルの **1列目（縦並び [x, y]**）を同じように掛け算して足します。

$$\begin{bmatrix} \cdot & \cdot \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow (4 \times x) + (-1 \times y) \Rightarrow 4x - 1y$$

この2つの結果を、上と下に綺麗に並べ直したものが、この掛け算の答えになります。

$$\begin{bmatrix} 2 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2x + 3y \\ 4x - 1y \end{bmatrix}$$

「左の手の人差し指で横にスーッと進み、右の手の人差し指で下にズーッと下がりながら、出会った数字同士を掛けて足す」というイメージです。

3. なぜエンジニアの世界で行列が必要なのか？

「横×縦の変な掛け算なのは分かったけど、これがどうして機械や電気的设计に役に立つの？」と思いますよね。現場での行列の最大の役目は、「めんどくさい連立方程式を、一瞬で片付けるための仕分けケース」になることです。

例えば、次のような中学・高校レベルの連立方程式を思い出してください。

$$\begin{cases} 2x + 3y = 8 \\ 4x - 1y = 2 \end{cases}$$

左辺の $2x + 3y$ と $4x - 1y$ って、さっきやった「横×縦」の掛け算の結果と完全に一致していますよね！
 ということは、この連立方程式は、行列を使うと次のように真ん中で引き裂いて書き直すことができるわけです。

$$\begin{bmatrix} 2 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

これを工学の世界では、それぞれ記号でまとめて次のような**究極にシンプル**な1行の式で表します。

$$A\vec{x} = \vec{b}$$

- A : 数字の仕分けケース（行列）
- $\vec{x}$: 求めたい答えのセット（ベクトル）
- $\vec{b}$: 右辺の数字のセット（ベクトル）

未知数が x, y の2つだけなら手計算でもできますが、実際の回路設計では未知数が $x, y, z, w \dots$ と5つも10も、時には数千個も同時に出てきます。人間には不可能なそんな計算も、この形にハメるだけで、PCが一瞬で解いてくれるようになります。

4. ワーク1：連立方程式をColabで一撃で解く

数式を $A\vec{x} = \vec{b}$ の形に整理できたら、あとはPythonにおまかせです。NumPyの `np.linalg.solve(A, b)` という魔法の関数を使います。

```
import numpy as np

# 行列 A を定義（横一行ずつカンマで区切って並べる）
A = np.array([
    [2, 3],
    [4, -1]
])

# ベクトル b を定義
b = np.array([8, 2])

# 連立方程式を一撃で解く（solve = 解く）
x = np.linalg.solve(A, b)

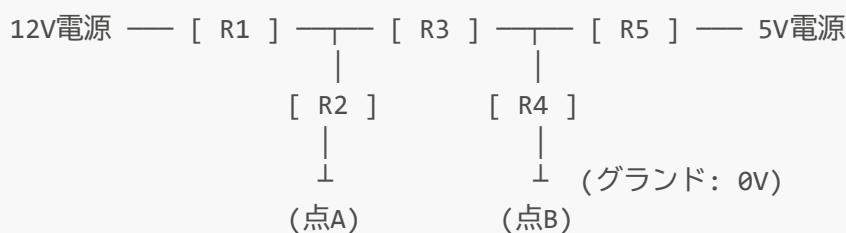
print("計算結果 (x, y) =", x)
```

🔗 出力の読み取り

実行すると `[1. 2.]` と出てきます。これは $x = 1, y = 2$ という意味です。中学生のとき、あれだけ苦労して文字を消去していた連立方程式が、一瞬で解けてしまいました。

5. ⚡ 【実務応用】複雑な電気回路の電圧を自動計算する

では、ここからが本番のエンジニアリングです。次のような、5つの抵抗と2つの電源が複雑に絡み合った回路（ブリッジ回路の親戚）の、**点Aと点Bの電圧** (V_A, V_B) を求めてみましょう。



それぞれの抵抗値は以下の通りとします。

- $R_1 = 100\ \Omega$, $R_2 = 300\ \Omega$, $R_3 = 200\ \Omega$
- $R_4 = 400\ \Omega$, $R_5 = 500\ \Omega$

この回路に流れる電流の通り道を「キルヒホッフの電流法則（ノード流出入の合計は0）」を使って整理すると、次のような V_A, V_B に関する強烈な連立方程式が立ち上がります。

$$\begin{aligned} \left(\frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3}\right) V_A - \left(\frac{1}{R_3}\right) V_B &= \frac{12}{R_1} \\ -\left(\frac{1}{R_3}\right) V_A + \left(\frac{1}{R_3} + \frac{1}{R_4} + \frac{1}{R_5}\right) V_B &= \frac{5}{R_5} \end{aligned}$$

分数だらけで、手計算で解こうとすると確実に計算ミスを誘う悪魔のような式ですが、これもさっき学んだ「仕分けケース」に入れてしまえば怖くありません。

6. ワーク2：回路の電圧をColabで求める

実際の抵抗値を当てはめて、行列 A とベクトル b に仕分け、Pythonに解かせてみましょう。

```
import numpy as np

# 抵抗値の定義 [Ω]
R1 = 100
R2 = 300
R3 = 200
R4 = 400
R5 = 500

# 行列 A (係数の仕分けケース)
A = np.array([
    [1/R1 + 1/R2 + 1/R3, -1/R3],
    [-1/R3, 1/R3 + 1/R4 + 1/R5]
])

# ベクトル b (右辺の数字の仕分けケース)
b = np.array([
    12/R1,
    5/R5
])

# 連立方程式を一撃で解く
V = np.linalg.solve(A, b)

VA = V[0]
VB = V[1]

print(f"点Aの電圧 VA = {VA:.3f} V")
print(f"点Bの電圧 VB = {VB:.3f} V")
```

実行結果

実行すると、点Aと点Bの正確な電圧が小数でパッと表示されます。回路シミュレータの裏側でも、全く同じ行列計算が行われています。

7. ワーク3：抵抗 R_3 を流れる電流を求める

点Aと点Bの電圧がわかれば、その間にある抵抗 R_3 を流れる電流 I_{R3} もオームの法則 ($I = \frac{V}{R}$) から簡単に逆算できます。 ワーク2のコードの最後に、次の計算を追加して実行してみましょう。

```
# 点Aから点Bへ流れる電流を逆算する
I_R3 = (VA - VB) / R3

print(f"抵抗R3を流れる電流 I_R3 = {I_R3 * 1000:.2f} mA")
```

このように、「複雑なシステムを連立方程式にする」→「行列の形 $A\vec{x} = \vec{b}$ に仕分ける」→「Pythonで一撃」という流れを覚えるだけで、あらゆる設計計算が全自動化できるようになります。

8. 【エクストラワーク】時間が余った人へ：教科書の問題を「瞬殺」する

早く終わった人は、教科書 pp.40 6, (2) を解いてみましょう。 とくに、手計算では地獄のような **3変数** (x, y, z) の**連立方程式** を Pythonで解いてみましょう。 行と列を増やすだけで一瞬で解けます。

$$(1) \begin{cases} x - y - z = 3 \\ 2x - y - 3z = 3 \\ x + 2y - 3z = -5 \end{cases}$$

$$(2) \begin{cases} x + y = 7 \\ 3y + z = -2 \\ x - 5z = 3 \end{cases}$$

```
import numpy as np

# (1) を解くコード例
A_extra1 = np.array([
    [1, -1, -1],
    [2, -1, -3],
    [1, 2, -3]
])
b_extra1 = np.array([3, 3, -5])

x_extra1 = np.linalg.solve(A_extra1, b_extra1)
print("解 (1) (x, y, z) =", x_extra1) # 答えは [2, -2, 1] になるはず！

# (2) を解くコード例 (x, y, z の係数がない部分は「0」にするのがコツ！)
A_extra2 = np.array([
    [1, 1, 0],
    [0, 3, 1],
    [1, 0, -5]
])
```

```
b_extra2 = np.array([7, -2, 3])

x_extra2 = np.linalg.solve(A_extra2, b_extra2)
print("解 (2) (x, y, z) =", x_extra2) # 答えは [3, 4, -1] になるはず！
```

【エンジニアとしての教訓：解けない式もある】 もし、全く同じ条件の式（例： $x + y = 2$ と $2x + 2y = 4$ ）を行列に入れると、Pythonは `LinAlgError (Singular matrix)` というエラーを出してキレします。これは「情報（条件）が足りなくて答えが1つに決まらないよ！」という警告です。PCが解けないということは、設計条件自体に欠陥があるという証拠になります。

9. 💡 【知的好奇心コラム】2次式（ x^2 や xy ）が出てきたらどうする？

今日の授業でやったNumPyの `np.linalg.solve()` は、**1次式（直線）専用の強力な武器**です。では、もし x^2 や xy のような **2次式（連立2次方程式）** が設計の中で出てきたらどうすればいいのでしょうか？

現場でのアプローチは大きく分けて2つあります。

① Pythonで「解く」なら：第4回のSymPyに戻る

2次以上の複雑なカーブ（非線形）を解くときは、第4回で学んだ **SymPyの `sp.solve()` ** の出番です。たとえば、次のような「円と直線の交点」を求める問題も、SymPyなら一瞬で解いてくれます。

$$\begin{cases} x^2 + y^2 = 5 & (\text{円}) \\ x + y = 3 & (\text{直線}) \end{cases}$$

```
import sympy as sp
x, y = sp.symbols('x y')

# 式 = 0 の形にして入力する
eq1 = x**2 + y**2 - 5
eq2 = x + y - 3

# SymPyのsolveに丸投げ
ans = sp.solve([eq1, eq2], [x, y])
print("解 (x, y) =", ans) # 出力: [(1, 2), (2, 1)] （交点が2つパツと出る）
```

② 数学的に「行列で表す」なら：左右から挟み撃ちにする（二次形式）

「2次式を行列で表す方法はないの？」というと、実はめちゃくちゃ美しい表し方があります。「**左と右から、ベクトルで挟み撃ちにする**」という特殊な合体技を使います。これを専門用語で『**二次形式（にじけいしき）**』と呼びます。

例えば、次の2次式を考えます。

$$2x^2 + 6xy + 5y^2$$

これを行列の形に仕分けると、次のように変身させることができます。

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

真ん中の行列の右上と左下に、 xy の係数（6）を半分ずつの「3」に分けて配置するのが美しく表現するコツです。

この「左右から挟み撃ちにする行列の形」は、将来みなさんが上の学年で学ぶ「AI（機械学習）の最適化アルゴリズム」、「3Dゲームグラフィックスでの物体の回転計算」、あるいは「機械・建築の構造の振動解析」などで主役として再登場します。頭の片隅に置いておいてください！

10. まとめ

今日登場した行列の武器を整理しておきましょう。

工学的な意味	数学の呼び名	Pythonでの書き方
ベクトルを並べたデータベース	行列	<code>A = np.array([[...], [...]])</code>
横方向の並び（漢字の形）	行（ぎょう）	前の行列から抜き出す成分
縦方向の並び（漢字の形）	列（れつ）	後ろのベクトルから掛け合わせる成分
連立方程式を一撃で解く関数	線形代数の解（solve）	<code>np.linalg.solve(A, b)</code>

来週の第15回では、この行列を使ったさらに実戦的な自動化シミュレーションに挑戦します！